

NYCU_DLP Lab6

1.Introduction

Given a specific set of label conditions, the DDPM should generate the corresponding synthetic images. For example, "red sphere," "yellow cube" and "gray cylinder" are given, the model should generate the synthetic images containing those objects. I also try different kind of beta schedule to observe the result.

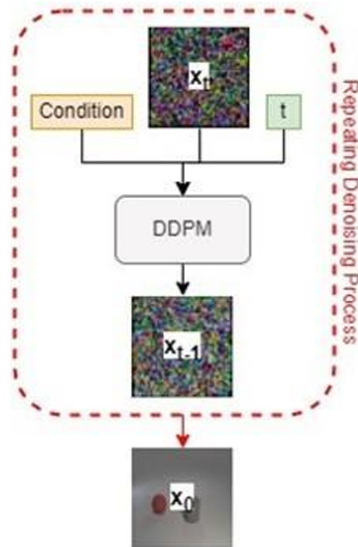


Fig. 1: The overview of conditional DDPM.

2.Implementation details

A.Describe how you implement your model, including your choice ofDDPM, noise schedule.

- The DDPM model:
 - I use the `diffusers` library's `UNet2DModel` as my main architecture with parameter in the following graph

- I overwrote the class_embedding with linear layer in order to implement the class embedding.
- The class embedding result will be added to the time embedding information to generate the time step information.

```
class DDPM(nn.Module):
    def __init__(self, num_class=24, class_emb_size=512):
        super(DDPM, self).__init__()
        self.unet_model= UNet2DModel(
            sample_size=64,
            in_channels=3,
            out_channels=3,
            layers_per_block=2,
            block_out_channels=(128, 128, 256, 256, 512,512),
            #norm_num_groups=32,
            down_block_types=("DownBlock2D", "DownBlock2D","DownBlock2D" ,"DownBlock2D","DownBlock2D","DownBlock2D","AttnDownBlock2D"),
            up_block_types=("UpBlock2D", "UpBlock2D","UpBlock2D","UpBlock2D","UpBlock2D","UpBlock2D","AttnUpBlock2D"),
        )
        self.unet_model.class_embedding = nn.Linear(num_class, class_emb_size)

    def forward(self, x, t, label):
        output = self.unet_model(x, t, label.float()).sample
        return output
```

- The noise scheduler:
 - I use the diffusers library's DDPMScheduler
 - It controls the addition and removal of noise, allowing the model to gradually learn how to generate or restore images from the noise.
 - By setting the time steps and noise scheduling method, you can control how the model handles noise at different stages, thereby influencing the final image quality and the model's training process.

```
noise_scheduler = DDPMScheduler(num_train_timesteps=args.time_step, beta_schedule=args.beta_schedule)
```

- The loss function, optimizer and scheduler:
 - **Loss function:** Mean Squared Error measures the difference between the noise predicted by the model and the actual noise. It calculates the squared difference between each pixel value of the predicted noise and the true noise, then takes the average.
 - **Optimizer:** Use the Adam optimizer
 - **Scheduler:** The scheduler is get_cosine_with_hard_restarts_schedule_with_warmup which coming from Hugging face's transformer library. This scheduler combines

Cosine Annealing and Hard Restarts, and it also supports learning rate Warmup at the beginning of training.

```
loss_function = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=args.learning_rate)
scheduler = get_cosine_with_hard_restarts_schedule_with_warmup(
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=len(train_loader)*args.epochs,
    num_cycles=50
)
```

- Training code:
 - We randomly sample noise and create a time step with a value between 0 and 999.
 - Input the image, noise, and timesteps into the `noise_scheduler` to add noise.
 - Input the noisy image, timesteps, and the label into the model to predict the noise.
 - Then we can calculate the loss and perform backpropagation.

```
for epoch in range(cur_epoch+1, args.epochs):
    train_loss = 0
    for img, label in tqdm(train_loader):

        img, label = img.to(args.device), label.to(args.device)

        noise = torch.randn_like(img) #sample noise
        #對不同的time steps 進行採樣
        timesteps = torch.randint(0, args.time_step-1, (img.shape[0],)).long().to(args.device)

        noisy_img = noise_scheduler.add_noise(img, noise, timesteps)

        output = model(noisy_img, timesteps, label)

        loss = loss_function(output, noise)
        train_loss+=loss
        #BP
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        lr_list.append(scheduler.get_last_lr()[0])

    scheduler.step()
    epoch_loss = train_loss/len(train_loader)
    loss_list.append(epoch_loss.detach().cpu().numpy())
    print(f'Epoch {epoch}: loss = {epoch_loss:.4f} : learning_rate={scheduler.get_last_lr()[0]}')
```

- Testing code:

- Randomly sample a noise `x`
- Traverse the time steps of `noise_scheduler`
- `Pred_noise` is predict the noise in the current image.
- The `noise_scheduler.step()` method applies the denoising operation of the noise scheduler. It adjusts the image `x` based on the predicted noise `pred_noise` and the current time step `t`, resulting in the denoised image `prev_sample`.
- Calculate the accuracy of the generating image

```
for label in tqdm(test_loader):
    x = torch.randn(label.size(0), 3, 64, 64).to(args.device) #sample一個全部都是雜訊的圖
    label = label.to(args.device)

    for i, t in tqdm(enumerate(noise_scheduler.timesteps)): # reverse process 0~999
        with torch.no_grad():
            pred_noise = model(x, t, label) #丟進noise predictor得到一張noise
            x = noise_scheduler.step(pred_noise, t, x).prev_sample #再透過noise還原原圖
            acc=evaluation_model().eval(images=x.detach(), labels=label) #最後一個time step 的acc

        #print(f"Time step {i} : acc {acc}")
        if i% 100==99:
            x_cpu=x.detach().cpu()
            for j in range(x_cpu.shape[0]):
                img_lists[j].append(x_cpu[j])
            #print(f"img_list has {len(img_list)} elements.")
            if i==999:
                last_img_list.extend(torch.unbind(x_cpu, dim=0)) #變成B個[C,H,W]
```

- Show result:
 - In the 0~999 timesteps,we store the 99,199.....999 timesteps result.
 - Store the denoise image step by step.
 - The last_image_list collect all of the final denoise result.

```

if i% 100==99:
    x_cpu=x.detach().cpu()
    for j in range(x_cpu.shape[0]): # 假设 x_cpu 是一个 numpy 数组或 PyTorch 张量
        img_lists[j].append(x_cpu[j])
    #print(f"img_list has {len(img_list)} elements.")
    if i==999:
        last_img_list.extend(torch.unbind(x_cpu, dim=0)) #變成B個[C,H,W]
        print(len(last_img_list)) # 輸出: 4
        print(last_img_list[0].shape) # 輸出: (3, 32, 32)
#print(f"The acc of {file}.json:{round(acc, 3)*100}%")

##### 最後一輪 產生圖片#####

for count, img_list in enumerate(img_lists):
    if not img_list: # 為空 跳出循環
        break
    tensor_images = torch.stack(img_list) #將列表轉換為四維張量 [N, C, H, W] 代表有幾張圖片
    grid_image = torchvision.utils.make_grid(tensor_images, nrow=10, padding=2) # nrow=5 表示每行顯示 5 張圖片
    grid_image_pil = transforms.ToPILImage()(grid_image)
    grid_image_pil.save(f'./img_{file}/output_image_{count}.png')

    tensor_images = torch.stack(last_img_list) #將列表轉換為四維張量 [N, C, H, W] 代表有幾張圖片
    grid_image = torchvision.utils.make_grid(tensor_images, nrow=10, padding=2) # nrow=5 表示每行顯示 5 張圖片
    grid_image_pil = transforms.ToPILImage()(grid_image)
    grid_image_pil.save(f'./img_{file}/final_result.png')

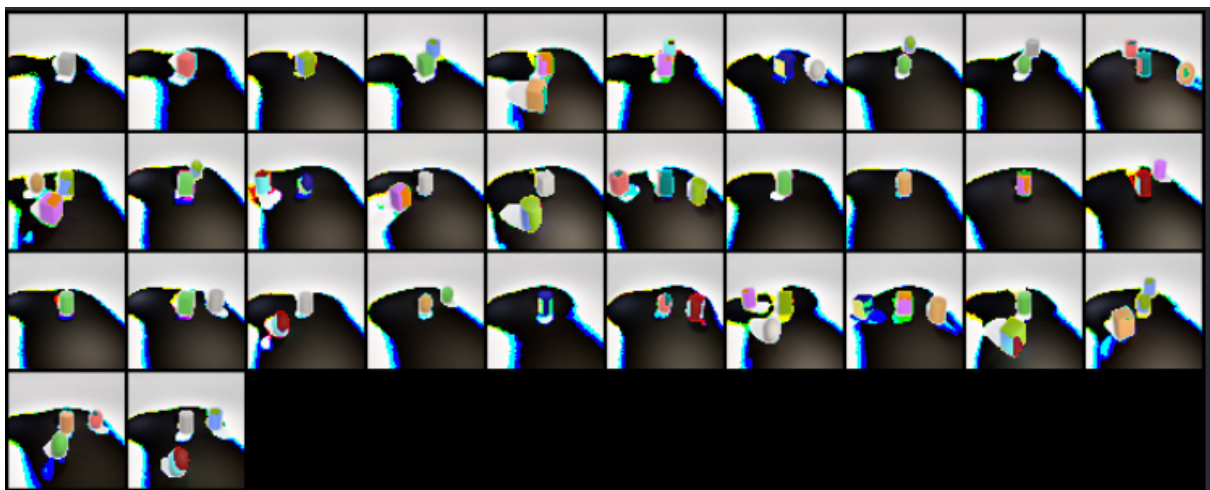
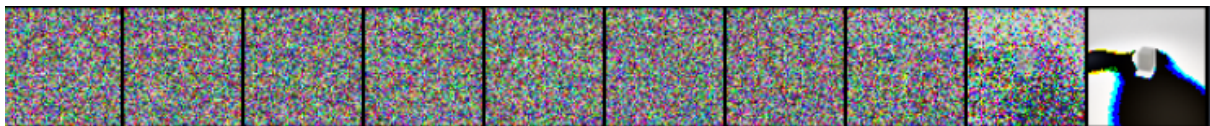
print(f"The acc of {file}.json is:{max_acc*100}%")
return round(max_acc, 3)

```

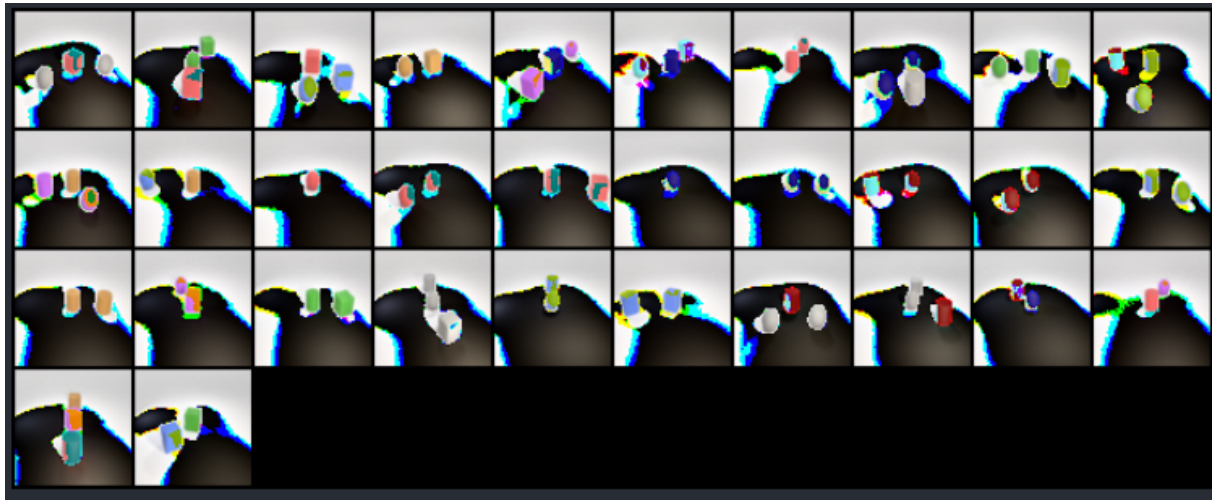
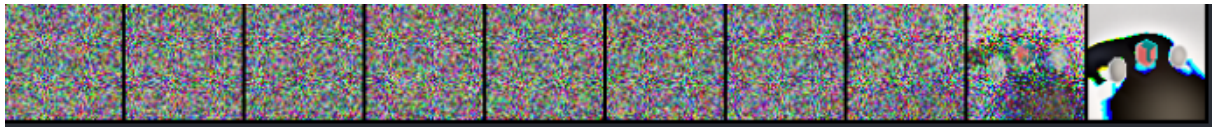
3.Results and discussion

A.Show your synthetic image grids and a denoising process image

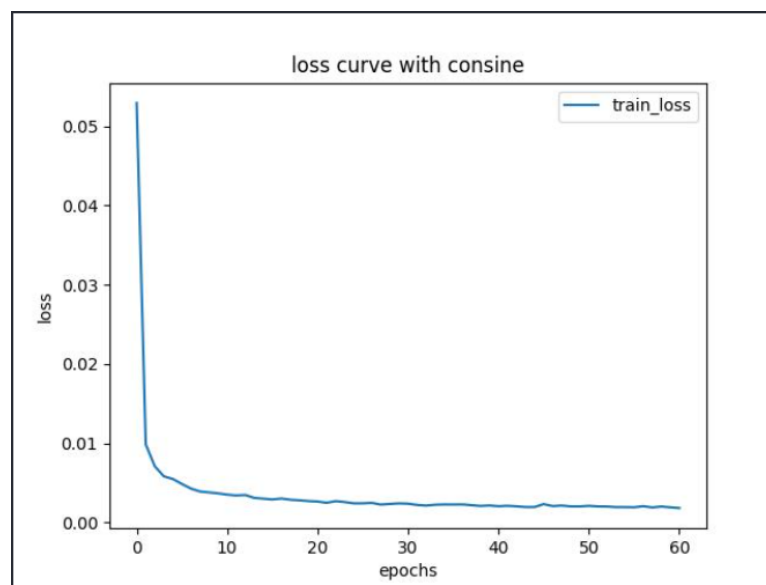
- Test result:



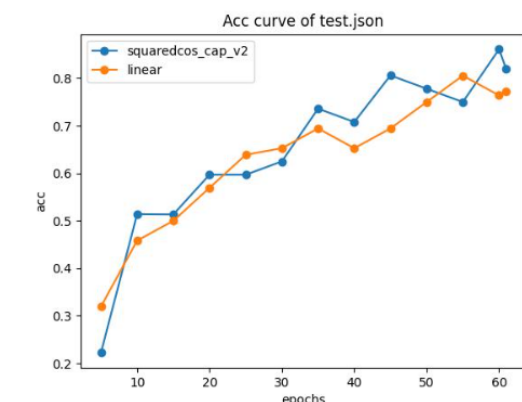
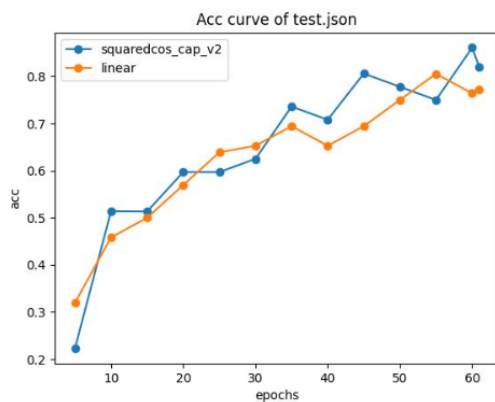
- New_test result:



- The loss curve:



- Accuracy of two testing file with epochs
 - We can notice that squaredcos_cap_v2 perform better



B. Discussion of your extra implementations or experiments

- **Experimentation with Different Beta Schedules:** Besides the linear beta schedule, I experimented with cosine and quadratic beta schedules. These experiments aimed to assess the impact of different noise schedules on the generated image quality. The results showed that the cosine schedule produced smoother transitions between time steps, leading to slightly better visual quality in the generated images.
- **UNet Variants:** I explored several variants of the UNet architecture by varying the number of layers, the width of each layer, and adding attention mechanisms at different scales. These modifications aimed to balance computational efficiency with image quality. In particular, adding attention layers at the bottleneck of the UNet improved the model's ability to generate high-resolution details.
- **LR scheduler:** Try different kinds of lr scheduler in order to result in smoother convergence and reduced the chances of overfitting.

Experimental results

1. Classification accuracy on test.json and new test.json

A.Show your accuracy screenshots

- The hyperparameter:
 - epoch : 61
 - batch_size :64
 - lr :1e-4
 - time_step:1000
 - beta_schedule:squaredcos_cap_v2
 - scheduler and optimizer

```
optimizer = torch.optim.Adam(model.parameters(), lr=args.learning_rate)
scheduler = get_cosine_with_hard_restarts_schedule_with_warmup(
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=len(train_loader)*args.epochs,
    num_cycles=50
)
```

- test: 80.55%

```
1000it [14:12, 1.17it/s]
100%|██████████████████████████████████████████████████████████████████████████████| 1/1 [14:13<00:00, 853.29s/it]
The acc of test.json is=80.55555555555556%
```

- new_test:80.95%

```
1000it [14:13, 1.17it/s]
100%|
The acc. of new test.json is=80.95238095238095% | 1/1 [14:13<00:00, 853.87s/it]
```

B.The command for inference process

- Make sure you are in file directory.
- The batch_size should be 64.
- --DS only can choose ['test','new_test']
- The experimental results accuracy may vary across different devices, leading to discrepancies.

```
python evaluate.py --ckpt /path/your_pth
--DS which dataset('test' or 'new_test') --save_path the
```

Ex:python evaluate.py --ckpt ./model_weight/Best.pth --save_path ./img --DS
test

Ref:

<https://huggingface.co/docs/diffusers/api/models/unet2d>

<https://huggingface.co/docs/diffusers/api/schedulers/ddpm>